Project Sprint #4

GitHub link: https://github.com/hafsaahmed05/CS449_Solitaire

This sprint assumes that all the features that support a human player to play a manual or automated Solitaire game is already implemented. The minimum features include **choosing the game mode (manual or automated), choosing the board size, setting up a new game, making a move (in a manual or automated game), randomize the state of the board** and **determining if a manual or automated game is over**.

The main task of this assignment is adding the feature of recording a game into a text file (or a lightweight database) and replaying from text file. The following is the complete GUI layout.

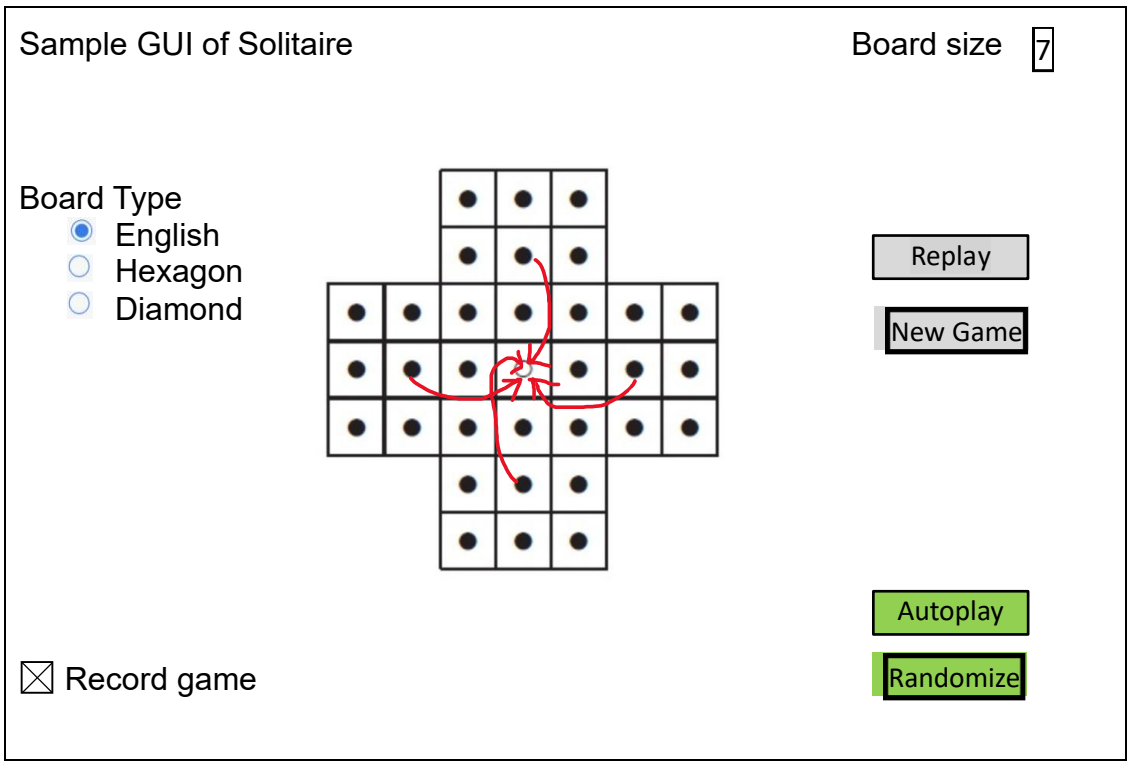

Figure 1: Sample GUI Layout of the Complete Solitaire Game

1. Demonstration (10 points)

Submit a link to a video of approximately 5 minutes, clearly demonstrating that you have implemented all the features in the following table. In the video, you must explain what is being demonstrated. No points will be given without a video link.

YouTube/Panopto link: https://youtu.be/6GdtC7i3q-I

	Feature
1	A complete manual game without using the “randomize” function is recorded
2	A complete manual game without using the “randomize” function is replayed
3	A complete manual game that sporadically uses the “randomize” function is recorded
4	A complete manual game that sporadically uses the “randomize” function is replayed
5	A complete automated game is recorded
6	A complete automated game is replayed

2. User Stories and Acceptance Criteria for the Record/Replay Requirements (5 points)

User Story Template: As a <role>, I want <goal> [so that <benefit>] Add or delete rows as needed.

ID	User Story Name	User Story Description	Priority	Estimated effort (hours)
9	Record Game	As a player, I want to record my game so that I can replay it later	High	2
10	Replay Game	As a player, I want to replay a recorded game so that I can review moves	High	2

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inPprogress)
9 story	9.1	AC 9.1 Start Recording Given the game hasn’t been recording yet When the user starts recording Then the recorder turns on and logs the board type and initial state	Completed
	9.2	AC 9.2 Record Moves Given the recorder is currently active When the player makes a move Then the move is saved in the correct format to the recording	Completed
	9.3	AC 10.1 Parse Recorded Moves Given a recorded move exists in the file When the system reads the move Then it correctly paeses the move	Completed
	9.4	AC 10.1 Record Randomize Event Given a recording Is active When the board is randomized Then the system records the randomized board	Completed
	9.5	AC 10.1 Recording Initially Off	Completed

		Given a new recorder is created When the recording has not started Then the recording should be inactive	
10 story	10.1	AC 10.1 Restore Board State Given a saved board state in a file When it is loaded Then the board is restored exactly	Completed
	10.2	AC 10.2 State Round-Trip Consistency Given a board state When it is saved and reloaded Then the state remains unchanged	Completed
	10.3	AC 10.3 Get Next Move - Valid Given a board with valid moves When the system requests the next move Then a legal move is returned	Completed
	10.4	AC 10.4 Get Next Move – No Moves Given a board with no valid moves When the system requests the next move Then a null value is returned	Completed
	10.5	AC 10.5 Replay Moves Correctly Given a recorded move When it is replayed Then it produces the same board state as the original move	Completed
	10.6	AC 10.6 Detect Invalid Files Given a malformed recording file When it is loaded Then the system detects the missing required fields and displays an error message	Completed

Carried over from Sprint 3:

Change	File(s)	Design Principle Addressed
Extracted GameRecorder as separate class	GameRecorder.java, SolitaireController.java	Reduces controller responsibility (Cohesion)
Moved file chooser dialogs to SolitaireView	SolitaireView.java, SolitaireController.java	Controller only handles logic, not UI (Cohesion, Coupling)
Replaced SolitaireModel int constants in CellButton with CellState enum	CellButton.java, BoardPanel.java	View no longer depends on model constants (Coupling)
Replaced instanceof HexagonModel with needsRowOffset()	BoardPanel.java, SolitaireModel.java, HexagonModel.java	View no longer depends on model subclass (Coupling)
Reorganized into model/view/controller/recorder subpackages	All files	Clear separation of responsibilities (Modularity)
Recording start/stop wired to checkbox listener	SolitaireView.java, SolitaireController.java	UI events handled in view, not controller startup (Cohesion)